

Programación de Servicios y Procesos

Modelo Productor-Consumidor

Cafetería

Autor:

Santi Martínez

December 3, 2025

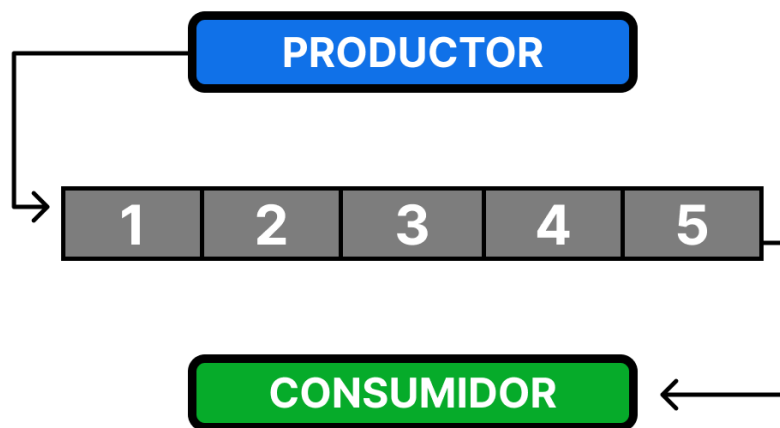
ÍNDICE

1	Introducción	2
2	Requisitos	3
2.1	Requisitos funcionales	3
2.2	Requisitos no funcionales	3
3	Modelo	4
3.1	Introducción del modelo	4
3.2	Buffer.java	5
3.3	Productor	6
3.4	Consumidor	7
4	Interfaz Gráfica	8
4.1	Pantalla inicial	8
4.2	Pantalla en ejecución	9
4.3	Pantalla final	10
5	Bibliografía	11

1 Introducción

Este es un programa en JavaFX en el que la principal tarea es jugar con los hilos (Threads).

La principal novedad en este programa es la implementación de un patrón productor-consumidor utilizando hilos y sincronización en Java. Para ello se han implementado los métodos **wait()** y **notify()** para gestionar el acceso concurrente a un buffer compartido entre productores (baristas) y consumidores (camareros).



En resumen, el barista produce cafés y los coloca en una mesa (buffer) con capacidad limitada. Si la mesa está llena, el barista espera hasta que un camarero recoja un café. Por otro lado, el camarero consume cafés de la mesa. Si no hay cafés disponibles, el camarero espera hasta que el barista prepare más.

Esta sincronización asegura que no haya condiciones de carrera y que los productores y consumidores operen de manera coordinada.

2 Requisitos

2.1 Requisitos funcionales

- Iniciar servicio
- Barista prepara los cafés y los deja en la mesa
- Camarero recoge los cafés de la mesa y los sirve a los clientes
- Mostrar en la interfaz las acciones del barista y el camarero en tiempo real
- Finalizar el servicio automáticamente cuando se hayan procesado los cafés determinados en la variable de configuración de cada clase
- Posibilidad de iniciar de nuevo el servicio sin reiniciar la aplicación
- Salir del programa en el momento que el usuario quiera

2.2 Requisitos no funcionales

- La interfaz se actualiza en directo
- La app es estable mientras se procesan clientes
- Funciona en cualquier plataforma que soporte Java y JavaFX

3 Modelo

3.1 Introducción del modelo

El modelo implementa el patrón productor-consumidor utilizando hilos en Java. En este caso, el barista actúa como productor que prepara cafés y los coloca en una mesa (buffer) con capacidad limitada. El camarero es el consumidor que recoge los cafés de la mesa para servirlos a los clientes.

En el software, se utilizan los métodos **wait()** y **notifyAll()** para gestionar la sincronización entre los hilos. Cuando el barista intenta dejar un café en la mesa y esta está llena, el hilo del barista se pone en espera utilizando **wait()**. De manera similar, si el camarero intenta recoger un café y la mesa está vacía, su hilo también se pone en espera.

Ejemplo

Este patrón, se utiliza por ejemplo cuando se realiza una suscripción a una web, newsletter, etc, donde dicho servicio nos manda un email. Al indicar el email y así como se clica en “suscribirte” el servidor, responde “Gracias por la suscripción” pero, a los dos minutos llega el email indicando dicha suscripción. Esto es porque se utiliza el patrón productor consumidor.

A la vista del usuario, al hacer click se ha registrado su email y ha llegado el mensaje a su email. Pero en realidad, el servidor ha puesto el email en una cola de mensajes y otro proceso ha ido leyendo esa cola y enviando los emails.

3.2 Buffer.java

Como su propio nombre indica, la clase que hace de buffer, compartida entre productores y consumidores. Alberga los siguientes atributos y métodos:

1. **CONTADOR_CAFES**: Número actual de cafés en la mesa.
2. **CAPACIDAD_MAXIMA**: Constante que define la capacidad máxima de la mesa (3 cafés).
3. **put()**: Método sincronizado para que el Barista deje un café en la mesa. Si la está llena de cafés (3/3), el barista espera hasta que un camarero recoja un café.
4. **get()**: Método sincronizado para que el Camarero recoja un café de la mesa. Si no hay cafés, el camarero espera hasta que el barista prepare más cafés y los deje en la mesa.

Estos métodos son fundamentales para la sincronización entre hilos.

- **wait()**: Hace que el hilo actual espere hasta que otro hilo lo notifique, normalmente con una condición. Libera el monitor del objeto, permitiendo que otros hilos accedan a él mientras está en espera.
- **notifyAll()**: Despierta a todos los hilos que están esperando en el monitor del objeto. Esto permite que los hilos puedan reanudar su ejecución cuando las condiciones cambien.

A nivel conceptual, el buffer gestiona la comunicación y sincronización entre el productor y el consumidor, generando un flujo controlado de producción y una iteración segura de datos entre los hilos.

3.3 Productor

En el modelo productor-consumidor, el productor es el hilo que genera datos y los coloca en el buffer para que el consumidor los procese. En este caso, el productor es el barista que prepara cafés y los deja en la mesa (buffer). En el caso de este proyecto, tiene las siguientes responsabilidades:

- Extiende **Thread** para ejecutarse de forma concurrente
- Realiza las iteraciones que se estimen en la variable de configuración en cada clase.
- Llama un método en el buffer **buffer.put()** para dejar café en la mesa
- Simula tiempo de preparación (aleatorio) con **Thread.sleep()**
- Imprime mensajes de estado en consola



Haciendo abstracción del contexto del café, el productor sigue el patrón clásico de producción en un sistema concurrente, asegurando que los datos se generen y almacenen de manera segura y eficiente para su posterior consumo por el consumidor.

3.4 Consumidor

La clase consumidor representa el hilo que consume los datos producidos por el productor. En este caso, el consumidor es el camarero que recoge los cafés de la mesa (buffer) para servirlos a los clientes. Sus responsabilidades son las siguientes:

- Extiende **Thread** para ejecutarse de forma concurrente
- Realiza las iteraciones que se estimen en la variable de configuración en cada clase.
- Llama un método en el buffer **buffer.get()** para recoger café de la mesa
- Simula tiempo de servicio (aleatorio) con **Thread.sleep()**
- Imprime mensajes de estado en consola



Al igual que el productor, el consumidor sigue el patrón clásico de consumo en un sistema concurrente, asegurando que los datos se procesen de manera segura y eficiente una vez que estén disponibles en el buffer.

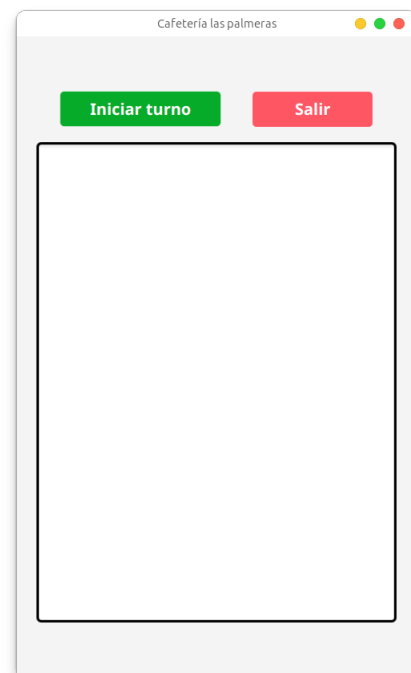
4 Interfaz Gráfica

4.1 Pantalla inicial

La interfaz gráfica está desarrollada con JavaFX y permite al usuario interactuar con el programa de manera sencilla. A continuación, se describen los componentes principales de la interfaz:

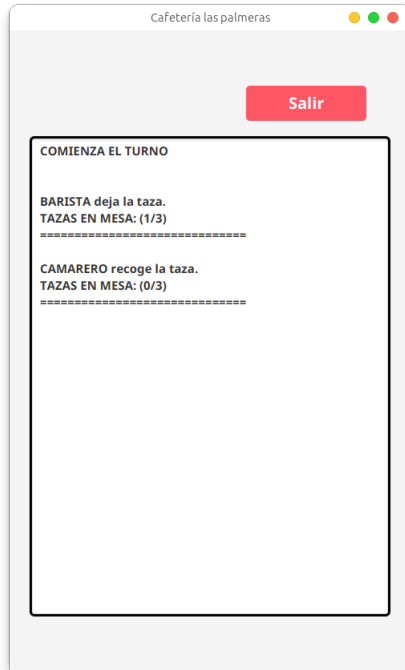
Botón "Iniciar turno": Al hacer clic en este botón, se inician los hilos del barista y el camarero. Este se desactiva mientras el servicio está en curso para evitar múltiples inicios. Al finalizar el turno, se reactiva para permitir iniciar un nuevo servicio.

Botón "Salir": Permite cerrar la aplicación de manera segura. Al hacer clic en este botón, se detienen todos los procesos y se cierra la ventana de la aplicación.



La interfaz se actualiza dinámicamente para reflejar el estado actual del sistema, proporcionando una experiencia interactiva al usuario.

4.2 Pantalla en ejecución



Botón "Iniciar turno": Desaparece o se desactiva mientras el servicio está en curso para evitar múltiples inicios. Al finalizar el turno, se reactiva para permitir iniciar un nuevo servicio.

Botón "Salir": Sigue permitiendo cerrar la aplicación de manera segura en cualquier momento.

Durante la ejecución, la interfaz muestra en tiempo real las acciones del barista y el camarero, proporcionando una experiencia interactiva.

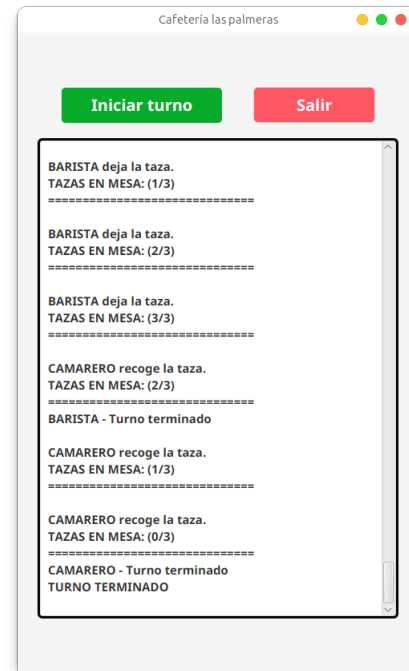
El programa **finaliza automáticamente** cuando se han procesado todos los cafés determinados en las variables de configuración de cada clase. Este final solo se produce cuando el camarero ha servido todos los cafés preparados por el barista.

Cada hilo realiza un número determinado de iteraciones (preparar y servir cafés) y una vez que ambos hilos han completado sus tareas, el programa concluye su ejecución.

4.3 Pantalla final

Botón "Iniciar turno": Reaparece o se reactiva para permitir iniciar un nuevo servicio si el usuario desea hacerlo.

Botón "Salir": Misma funcionalidad que en las pantallas anteriores, permitiendo cerrar la aplicación de manera segura.



Al finalizar el turno, la interfaz muestra un mensaje indicando que el servicio ha concluido. El usuario tiene la opción de iniciar un nuevo turno o salir de la aplicación.

5 Bibliografía

- Uso del PrintStream: stackoverflow.com